Juha Laurila/ Devops-blog
===

Introduction to my Devops blog

Hi!
I'm Juha Laurila from Kajaani. I have really long work experience of all kind of marketing communication. About a one year ago I was obliged to leave my job in Jukkatalo because of decreasing sales caused by effects of coronavirus. After that I decided to continue and intensify my ICT studies because of weak employment situation of marketing communication field.

H2C - top story to improve ICT- and programming skills!
When I heard first time about this H2C-course I thought, referring to the contents of the course, it might be really big benefit to me to learn more current methods in software development in general and also a few specific things such as basics of python, databases and modern ways to do www design with fullstack methods. Although learning new interesting things has always been near my hearth, the most ultimate aim - also with these H2C-studies - is to increase my opportunities to get a job in ICT sector, of course.

Module 1 - Development tools for teams and individuals

Before this course, I have not any experience of Git and Slack. Third stronly rising development method, Docker, was already a little more familiar to me because of using it in H2C python course earlier last winter. Anytway, I was very unaware of these modern possibilities and methods to rationalize, simplify and speed up software development process both at the individual and group work. The most important purpose of Devops course is to learn use these modern tools.

Using Jupyterlab in Docker environment, I have not quite a clear understanding of Docker yet. After I read more about Docker, containers and all universal things about that technics, my eyes opened finally to see benefits of Docker to do many things easier, faster and with lighter resources and bigger agility.

Agility and resource saving technology to development with Docker
Using containers enable better portability of things, because containers includes all necessary elements to compile and run an application in all-in-one standardized packet - container. Compared traditional way to build a server for certain application, containers save really much hardware resources, because they need not own operating system to work. In other words, same server computer can run really many containers on the same operating system. In addition, containers decrease challenges of developers with difficult operating systems because same container works as well in all systems.

Agility to communication with Slack
Slack was a second thing, which proved its usefulness almost immediately at the beginning of the course. Compared to email, communicating by slack channel is much more effective and faster, whether it's a conversation with an individual or a group. In addition, all the issues, posts and information are in same place, easy to find, benefiting all the users of the channel. E-mail is really graveyard of information!

Agility to version management with Git
The last but not the least in this list of developing tools is Git. I have heard many times of Git earlier, but it has been covered by shame, until Devops course began. Learning basics of it gone at the same way as many other thing related to computers and software earlier. Again, there is only a one method to learn: read, ask at a low threshold more and understand instruction and finally, do things carefully according examples.

Juha Laurila/ Devops-blog
===

So, after installing Gitlab and obligatory Git bash (because there is not suitable command line user interface in Windows naturally), I had eventual all tools available. First step was to understand how the Git works altogether. Principles of the function and many anvantages of the Git were quite easy to notice and understand. Second and bigger challenge was learn to use necessary commands and manage whole program in practise.

Doing according to instructions I succeed to create new project, include ssd-keys to it and clone remote repository to my local laptop for further processing. After few training session I noticed, that it is not so difficult at all .

Module 1 - Tasks 1-7 assigments completed

- Task 1: Register to Gitlab at gitlab.dclabra.fi : Done
- Task 2: Create your own git project: Done
- Task 3: Register to wakatime: Done
- Task 4: Install Visual Studio Codet: Done
- Task 5: Set up wakatime time tracking: Done
- Task 6: Install git: Done
- Task 7: Set up your personal blog and briefly introduce yourself: Done

Module 2 - Introduction to Agile Distributed Software Development

Task 8: Scrum roles and responsibilities

- Product owner (PO)

In the scrum team, one person is the product owner who is responsible for maximating the value of product and development team work. The product owner is familiar with products and knows the product's business and represents customers and users. The product owner supervies and ensures that development team has focused to do things, which are the most important ones for product development just in that sprint. In the sprint review, the product owner approves the product version of the previous sprint and participates in the sprint design meetings.

- Scrum master (SM)

The most important task of scrum master is supervise and ensure, that all the members of scrum team are understood scrum rules in theory and in practice and also uses it, in their everyday work. Scrum master has not direct managerial position in relation to members of scrum team, instead that SM effects throught the scrum process, communicating with development team and product owner, if the situation so requires, for example, achieving sprint goals is starting to look unlikely.

- Developer (Dev)

Developer is a member in multi-skilled development team, which is third part of scrum team entirety. Only developers participates to the actual product development process. Every developers can have different skills and priorities in development team. However, the responsibility for development lies with the entire development team together.

A few findings (examples in Palkkavertailu.com) of average salaries:

Product owner, 4 403 €
Scrum master, 4 019 €
Software developer, 3 524 €
Design engineer hw, 3 700 €

A quick resultat is that PO and SM are valued more compared to single developer, surely salaries of developers vary considerably depending on the company, job, place and experience.

Task 9: How to utilise Scrum in practise?

Short story with scrum

Eventually, after many days waiting for something bigger to happen, the phone rang in the room of sales manager. In fact, he has not only sat waiting for that in his byro, there was everything to do, almost always. But the call he got was very big and important, because it confirmed that the offer sent by himself to significant customer had accepted. After that, it was time to priorize plans for next months because the customer was willing order a new accounting software. And in addition, key person of the company have decided to start using scrum methods if they will get an order from this customer. They felt, something really big thing was happening at that moment. They got to show their nails!

Organization to scrum
The time had come to organize for Scrum. It was also time to determine things accurately in practice as objectives, requirements and timetable of the project. The executive team of the company met for the first planning meeting.

After meeting they had efficient and effective organisation for that particular project but also for lately use, if the organization would work well.

Product owner and Product Backlog
Topi M. was appointed product owner, because he was familiar with accounting programs and also worked many years with corresponding positions in several companies. His first task was to organize a Product Backlog which includes a list of everything that may be needed in the product. The list will be also only source for requirements and changes to the products.

Scrum master
As a scrum master was elected Tiina K. with one accord. She has been staff manager earlier and also known as aa accurate and responsible person. In addition, she has worked before as a scrum master in other company. From the beginninng of the sprint, scrum master took care of working of development team that all will be done according to scrum methods.

Development team
Development team was mostly ready already, only one or two arrangements had be to done. Pekka P, product development manager of company, was natural choice also to lead development team in this project. Eventually, there were now sufficient resources and widely skills, and the whole development team was motivated to take up the challenge.

Customer and Requirements
There were many requirements with the accounting program ordered by customer. The customer, in principle, had determined that the most important, obligatory features of the new program should be scalability, simple intuitive user interface, database and customer management properties. Naturally, all should to work via www, with cloud implementation.

All the obligatory requirements were accepted earlier, only some options which customer presented had be left outside of the offer, to be decided as a possible additional order later.

Sprints and Sprint Backlog
The developent team met to the first sprint meeting.
All the requirements were written in Product Backlog and team decided under the supervision of product owner to implement first release merely to testing all techniques needed, by the end of the first sprint period. These things was recorded to Sprint Backlog, and according it the first sprint should be proceeded. In other words, certainty about technical operation was the most important requirement of the first sprint and first release of the program.

Development and evaluating of the product and past sprint
After three weeks they had a simple release ready to test. The development team has widely skills and competence to solve every sub-area in th sprint. The product owner ensured, that every sub-area worked perfectly. After succesful testing it was time to test the whole chain. The result was that all the sub-areas worked also together.

At the end of the first sprint period all the achievments of the project were evaluated in sprint review. Before the start of the next sprint, a retrospective of the sprint was held to review the past sprint process and possible areas for development to be implemented in the next sprint.

Iterations and increments - step by step to more ready product
One essential part of scrum process is iteration, there was not an exception in this case. After the second sprint planning meeting they have a clear plan to implement actual functionalities of accounting business to their first sceleton program release. Again, all the most important requirements were upwritten to Sprint Backlog of current sprint, which is a part of bigger entity - Product Backlog. So, as the project progressed it was added new properties and more complex structures, as determined in current Sprint Backlog.

At the end of second sprint, thay have again a well working but more anvanced program with increments determined by second sprint backlog. After evaluating, that new program release corresponds to the finished definition in that sprint, it is time continue again to following sprint with all scrum meetings, instructions and so on, again and again until the program had achieved the final level required and accepted by the customer.

Certainty, predictability and efficiency with several well done sprints
In this case, there were five sprints in all - separate periods of four weeks. Each of them has an individual plan to determine degree of readiness at the end of sprint and also to implement these most important things in practice, under supervising of scrum master and product owner, that requirements of the current sprint will be achieved. Eventually, after sufficient times of sprint iterations, the whole product will be finally ready to be handed over to the customer.

Module 3 - Docker for DevOps Teams
Task 10: Docker for distributed development teams

Well, today was again one of my nice days, eventually. I managed to get Agar.io clone to work, using Dockerizing, supplemented by little adjustments to the codes, according to instructions! The easiest task in this project was using Git, with whom I am alreay quite familiar, at least with its basic use as cloning a project from remote repository to my own local one, creating a new project to remote repository, using add, status, commit, push and pull commands with Git Bash command line tool in Windows operating system. There has been really much benefit to me in using command line to completing the courses "Linux OS" and "Embedded c Linux" earlier with my other laptop, with Ubuntu 20.04. Git is top system in agility and distributed software development, with all its useful properties (release management, increased transparency in working of teams with same project, pipeline, issues, epics etc.)

But how about Docker?
I can say, it was neither not hard to learn principles using it. Operations with it happens also using Git Bash command line. Im sure, based on the experience of this course and also what I have read on the subject, that Docker is equally important tool improving performance and compatibility. In addition, it helps to reduce costs by decreasing need for hardware resources (one server computer can offer much more services with dockerizing, with containers, compared to traditional way of doing these things. In addition, software development will be much more agility, faster and more effortless, when services are in containers, because things will be easily tested and implementation of services is easier.

One of the biggest benefit in connection with Docker is independency of the operating system of developers. When software development is done with containers, all the developers have similar developmen enviroment, regardless of the current operating system. This decreases significantly problems, challenges and reduces duplication and unnecessary things in software development.

So, what about the challenges of the task? Well, iteraton is always mother of studies, also in this case. At the beginning, there are many unknown things, many cause consequence relationships, and knowledge is there and here. First, I need to know well enough at a general level what is going on. After that it will be easier to centralize to essential things, such different files, operations and commands tu built up container and necessary elements to implement a working application. But, focus on the topic and materials, repeats and iteration, this is the Thing!

Module 4 - Agile Project Management with Gitlab
Task 11: Functional requirements with UML Use Case diagram

This figure shows detailing what choices the user can make before the game and what the player should aim for in the game itself.

This figure shows much more simplified, on a general level, key use cases in the game.

Although I am a novice agar.io player, it has now become even tested a little. As a player roles, in addition to the beginner as I am, there can be lot of different player types such as an average single player, experienced single player, a logged in player, team player, friend player, a player who plays for political influence etc. Everyone has a slightly different role and motivations to play.

When looking at the UML chart, there can be seen many general steps and requirements that a player, regardless of role, must follow in order to succeed in the game. In addition, if a player is somebody other than basic player, there are additional options and also new requirements for them, of course.

Mandatory general requirements and aims for every players to succeed in the game are:
1. Eat balls smaller than yourself that you will grow and get points
2. Don't be eaten by other, bigger competitors
3. Don't collide with a virus if you're bigger than that

Many roles of players
If you are a beginner and first time playing, is the most important requirement to player only to test and to try different alternatives and properties of the game at a low threshold, taking it not too seriously at first. Through this approach learning a new game is faster and more fun. More advanced players can then focus more seriously on scoring and success, and so requirements to manage in different situations are naturally

harder.

When the game is played at first time, it learns effectively by intuitive way - its very easy to notice, that when the player's ball hits the smaller ball, at the same time players ball grows a little and the score increases. Also, when the player's ball hits the bigger ball compared to players ball, happens destroyment of player and the game ends and so on. Requirements to understand an ultimate idea in the game principles are therefore not very difficult to anyone.

UML chart clarifies the relationships of things in the game
In the UML chart, there have drawn by solid black lines with arrays between the boxes, which are essential requirements for all players, independ of a player type or role.
Things, which are additional properties and not necessary requirenments get to play the game and in the in gaming in general, have connected with green line.

Some operations can be mandatory requirements to get play, although they are not directly related to the game itself. In this game, there is the robot testing which falls into this category, indicated by red solid lines with arrows. To pass the test, a player has to solve image test to prove that he or she is a human, not a robot.

Optional properties and requirements for them
Most of optional properties will be chosen before to game. In the UML chart, these boxes has been connected by black dash dot lines. For example, if you want to get nickname or skin to your game character, you must give a name and choose skin before starting a game. If you want to get more alternatives to buy and use different skins, the requirement for this is logging with your Facebook or Google account. Althought you have signed in, its not possible to buy new skins before you have succeeded and got enough experience points and coins in the game.

As a team player, one of the basic requirement is suitable team, of course, with whom to play. Secondary, but also very important is a suitable name and skin to make the team stand out well in the game area. In order for a team and a friendly game to be successful, there must also be some channel of communication between the players - chat, phone and so on, if the players are not in the same room.

When different players play the game, all of them have slightly different goals in gaming. That is why there are so much different roles with players in practice - so I think that categorizing them to a few clear groups is always a bit artificial eventually, maybe.

Module 5 - CI/CD in Gitlab

Task 12: Final Assignment

- Preparations
-- Slack notifications
-- Ignore unnecessary files with .gitignore

According to video, I got the first thing to completed - to connect my Gitlab repository with Slack (Add incoming webhook) to channel #final-assignment-notifications.

Then was a time to clone my Git repository into my local laptop.
After that i created .gitignore file to the local repository and made the routine procedure to be transferred it to my remote Git repository (with commands add, commit and push).
Then I had all the preparations for the project ready.

- Part 1 - Issue management (i.e. requirements)

Then I made a few operations in Git, related to issue management.
I created three issues by relevant names and added labels to them.
Then, according to instructions, I began to create all the files needed to implement working web based temperature and humdity measurement application.

- Part 2 - creating the HTTP server

At first, I created package.json file with contents and repeat all the steps as mentioned above with .gitignore file to getting the new file succesfully to my Git repository. The same way I did with the server file.

- Part 3 - Reading the sensor

To make possible reading tempertures and humidities of dht sensor, it has to be created a code implementing these operations. After creating the code it had be sent by same methods than codes also earlier, by Git Bash commands.

- Part 4 - Dockerizing an application

Then it was the dockerization's turn. I created a Dockerfile with specified content. After that I build a container, but at first I launched Docker application. Building process took quite long time in spite of its desktop lab compiling, but eventually it became ready. When I run the builded container, everything seemed to go as it should. Temperature and humidity values began to appear in the shell, with a little delay between of them. The name the Docker had given to container, was someone random default one, because i have not given any specialized name.
At final, I push again just created Dockerfile to Git with comments.

- Part 5 - Automating unit tests

When dockerizing has implemented succesful, its time to do required operations to create automatized unit testing prerequisites. First of all, I created a new directory called test. Inside of this directory I created a new code with functiond for two unit tests, with whom could be tested availabilities of two different URL. Again, changes have to be sent to Git, too.
Before run the test, I should yet install all the dependencies with npm install command and then the actual test by npm test command. Result of the test showed, that both of the URL:s was available, information of the dht sensor has read, testing times has also reported and overall, print of the run looks like an example print.

I tested also to run test in the container:
-- 1. docker build -t test/weatherserver .
-- 2. docker run -it test/weatherserver npm test

Both build and run commands succeeded as instructed.

recting and repushing it to git, pipeline succeeded and there were shown three stages in Git: built passed, test passed and finally, deploy stage for possible manual approval and deployment of tested code, for example.

- Part 6 - Tweaking Gitlab CI/CD pipeline

In this task, one of the most important objectives was to implement automatized pipeline workflow in Git. To be reaching that, I had to create all te required files with relevant contents according to instructions After that I push all the required files to Git repository. When all the files were in the remote repository succesfully, it was a time to implement pipeline functionality to it. .gitlab-ci.yml was in an essential role triggering pipeline to run and building all necessary stages to it.
At the beginning of the test, I have a little but significant problem with missing .yml file extension. After correcting and repushing it to git, pipeline succeeded and there were shown three stages in Git: built passed, test passed and finally, deploy stage for possible manual approval and deployment of tested code, for example.

- Task 7: Read sensor data from the browser

In the last task I implemented sensor measurement function via www,
using generic open source automation framework - Robot Framework.
The first step to learn was get to know the subject. In the materials given by Devops course, there was useful example of implementing a simple solution to check the validity of signification. When I had downloaded the example project to my laptop, I created a new project to Git for this latest task. Then I cloned the project to my local workstation and added those example files to it.

After that I pushed files to remote repository and tested the function of example project. All worked well! To join temperature and humidity measurement from https://dwbl.dclabra.fi/api/v1/dht1, I modified the index.html file a little - I changed simply the link caused succesful logging to link of page of dclabra mentioned above, instead of previous Wellcome page. Now, it worked well, in principle. Of course, if the measurement information only would be as an imported and isolated element inside the page layout, finished with css styles and so on, it would seem better, of course!